

Example problem – Linked list

Sistemas de Computadores

Departamento de Engenharia Informática

Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

Overview

- Single lock
- Single reader-writer lock
- Multiple reader-writer locks

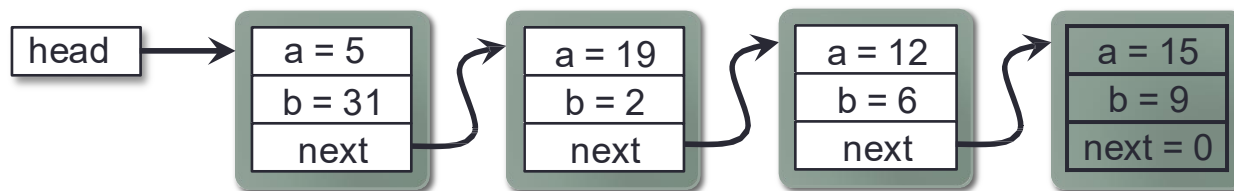
Example problem: Linked list

- How would we implement a linked list that supports concurrent access from multiple processes/threads?

- Operations on our linked list:

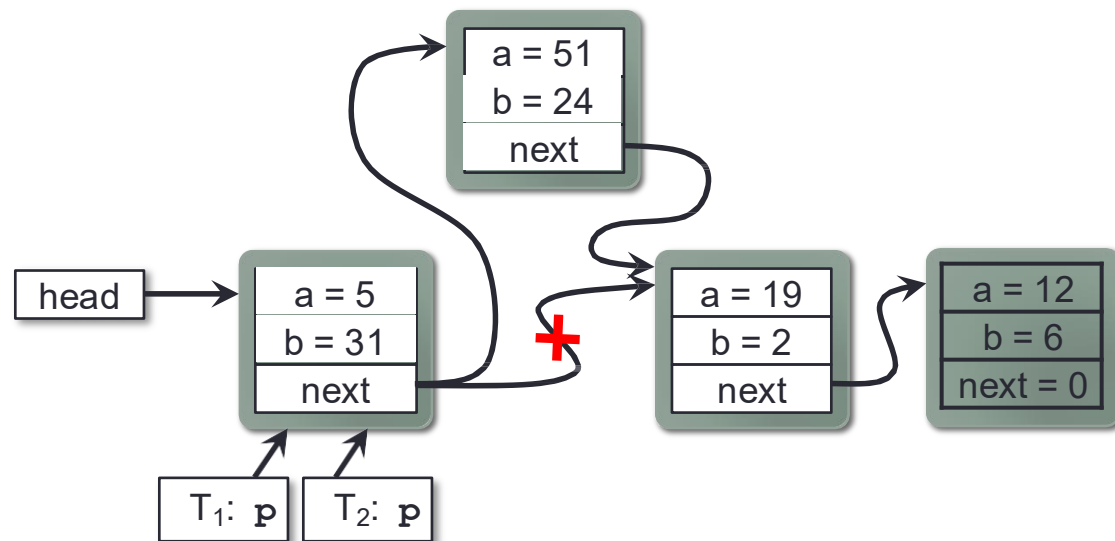
- Iterate over the list nodes
- Insert a node into the linked list
- Modify a node in the linked list
- Remove a node from the linked list

```
typedef struct{  
    int a;  
    int b;  
    struct list_node *next;  
}list_node;  
  
list_node *head;
```



Linked list and concurrent access

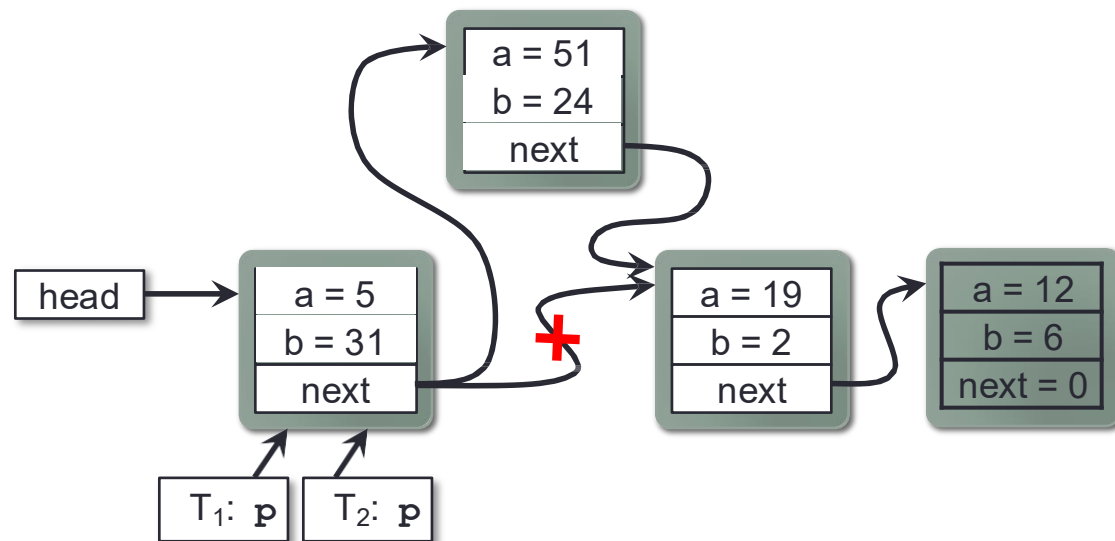
- By now it should be obvious that our linked list will be corrupted if manipulated concurrently by different processes/threads
- **Example:**
 - One thread T_1 is traversing the list, searching for the node with $a = 12$, so it can retrieve the current value of b
 - Another thread T_2 is inserting a new node into the list



Linked list and concurrent access

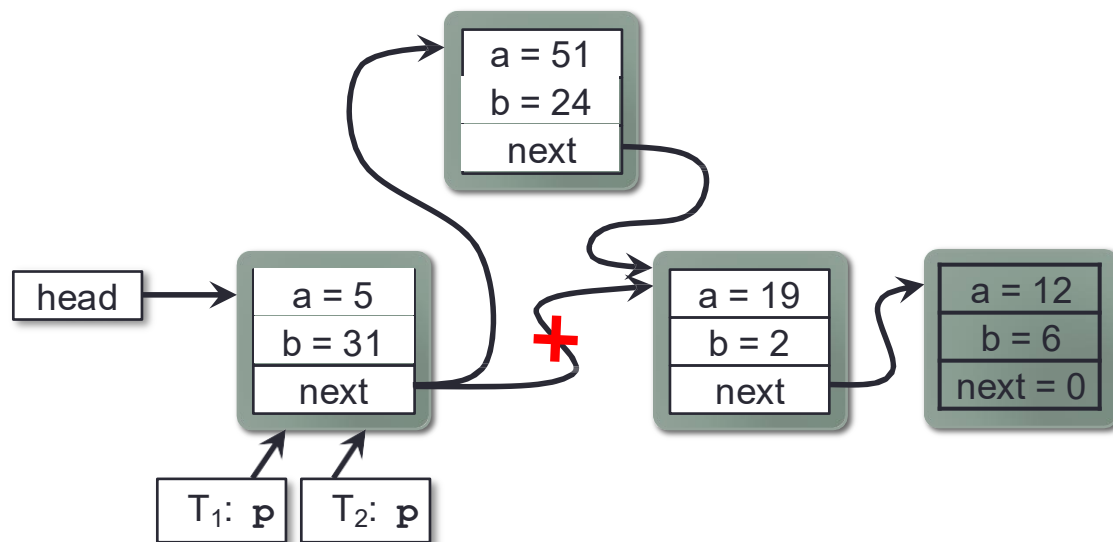
- **Writer-thread T2 must perform several operations**
 - This scenario can fail in many different ways

```
list_node *new = malloc(sizeof(list_node));  
new->a = 51;  
new->b = 24;  
new->next = p->next;  
p->next = new;
```



Linked list and concurrent access

- These operations form a **critical section** in our code
 - We can try to specify a reasonable order of operations
 - But really have no guarantees about how the compiler will order this (or the CPU, for that matter)
 - Must enforce **exclusive access** to the affected nodes during these operations



Linked list: A single lock

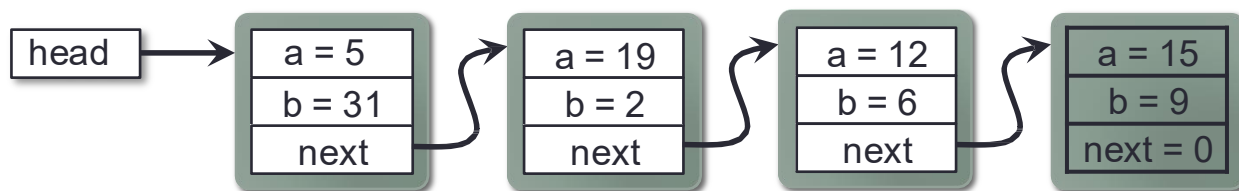
■ An easy solution:

- Use a **single lock** to guard the entire list
- Any thread that needs to read or modify the list must acquire the lock before accessing head

■ Why must readers also acquire the lock before reading?

- Only way for the writer to ensure that readers won't access the list concurrently while it's being modified

```
typedef struct list_node{  
    int a;  
    int b;  
    struct list_node *next;  
}list_node;  
  
list_node *head;  
pthread_mutex_t list_lock;
```



Linked list: A single lock

■ What's the obvious issue with this approach?

- Readers shouldn't block other readers
- It's okay if writers hold exclusive access to the list while modifying it
- (it would be better if multiple writers could concurrently modify independent sections of the list)

■ Has very **high lock contention**

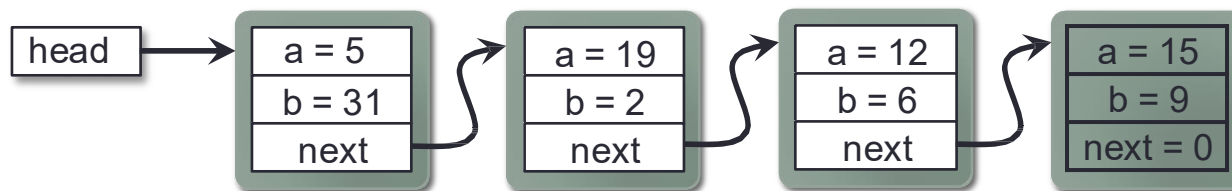
- Threads spend a lot of time waiting to acquire the lock so they can access the list
- No concurrent access is allowed to the list

Linked list: Using a read-write lock

■ Use a read-write lock instead of the mutex lock

- Readers can access the shared resource concurrently without any issues
- Writers can acquire exclusive access to the lock
- (we will accept the behavior that writers block everybody, for now)

```
typedef struct list_node{  
    int a;  
    int b;  
    struct list_node *next;  
}list_node;  
  
list_node *head;  
pthread_rwlock_t list_lock;
```



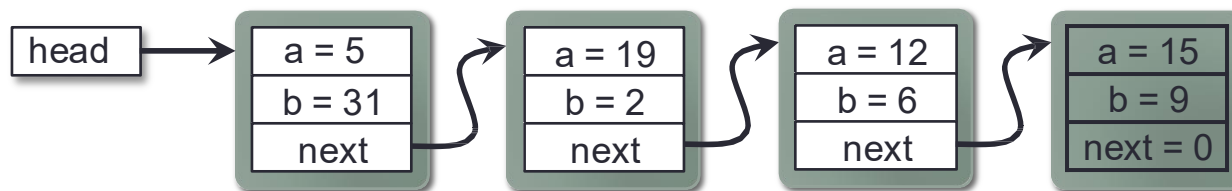
Linked list: Using a read-write lock

- **Using a single read-write lock increases concurrency and reduces lock contention**
 - However, still a few annoying issues remain
- **Readers still must acquire a lock every time they access the shared resource**
 - All threads incur a certain amount of lock overhead when they acquire the lock (in this case, CPU cycles)
 - It turns out this overhead can be hundreds of CPU cycles, even for efficiently implemented read/write locks!
- **Also, writers still block everybody**

Linked list: Multiple locks

- Reduce lock contention by decreasing the granularity of the lock
 - i.e., how much data is the lock protecting
- Introduce **more locks**
 - Each of which governs a smaller region of data

```
typedef struct list_node{  
    int a;  
    int b;  
    pthread_rwlock_t node_lock;  
    struct list_node *next;  
}list_node;  
  
list_node *head;  
pthread_mutex_t list_lock;
```

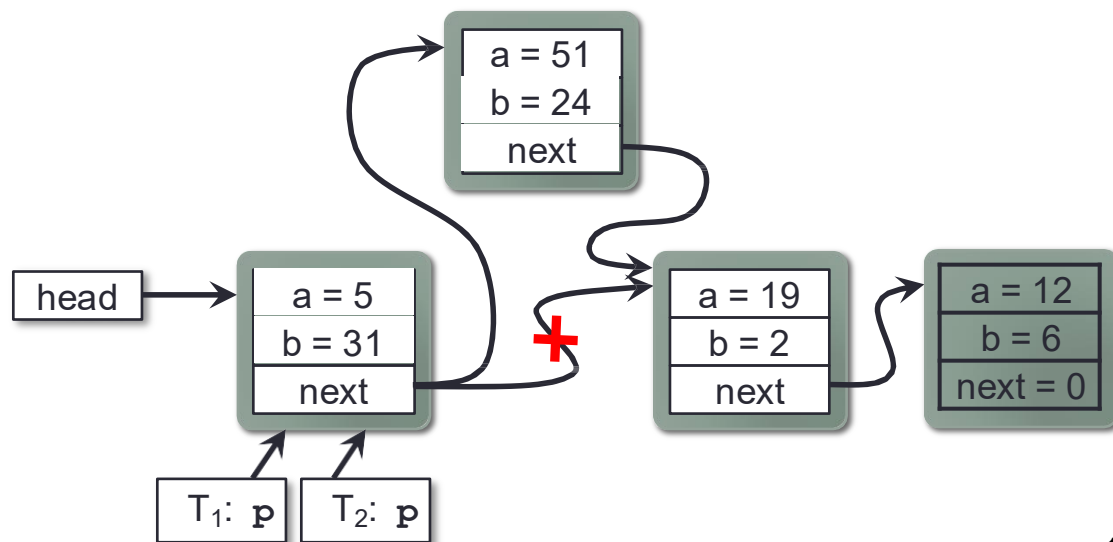


Linked list: Multiple locks

- **Threads must acquire many more locks to work with the list**
 - Which means that the locking overhead goes way up
- **But writers can lock only the parts of the list they are changing**
 - Which means we can reduce lock contention/increase concurrency
- **With multiple locks in our code, must beware of the potential for deadlock**
 - Can easily avoid deadlock by requiring that all threads lock nodes in the same total order (break the “circular wait” condition)
 - This is easy – it’s a singly linked list! Always lock nodes in order from head to tail

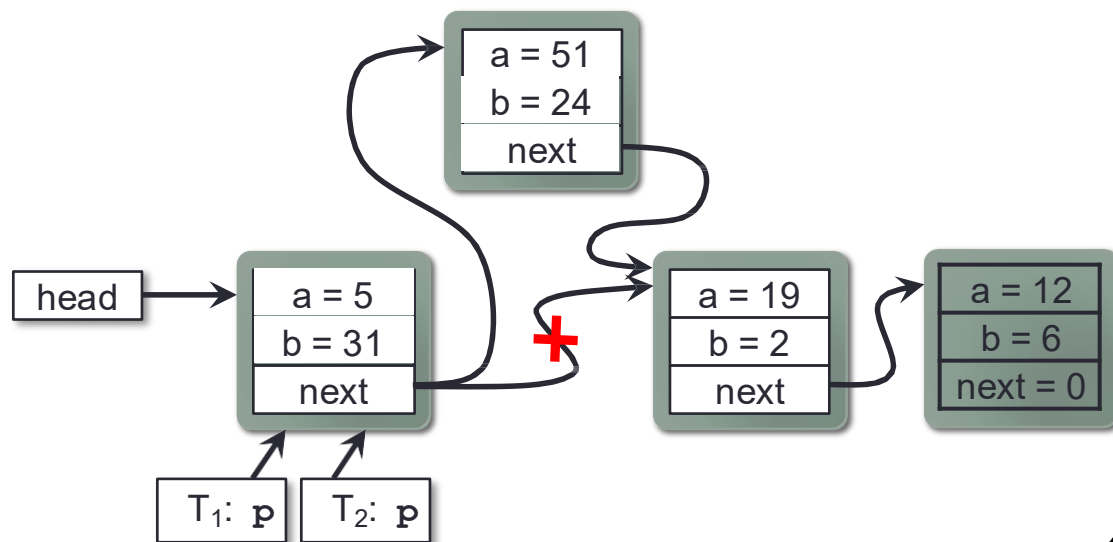
Hand-over-hand locking

- **T_1 acquires a read-lock on head so it won't change**
 - Then T_1 follows head to the first node, and acquires a read lock on this node so it won't change (and so forth)
 - As long as T_1 holds a read lock on the current node, and acquires read-lock on the next node before visiting it, it won't be affected by other threads



Hand-over-hand locking

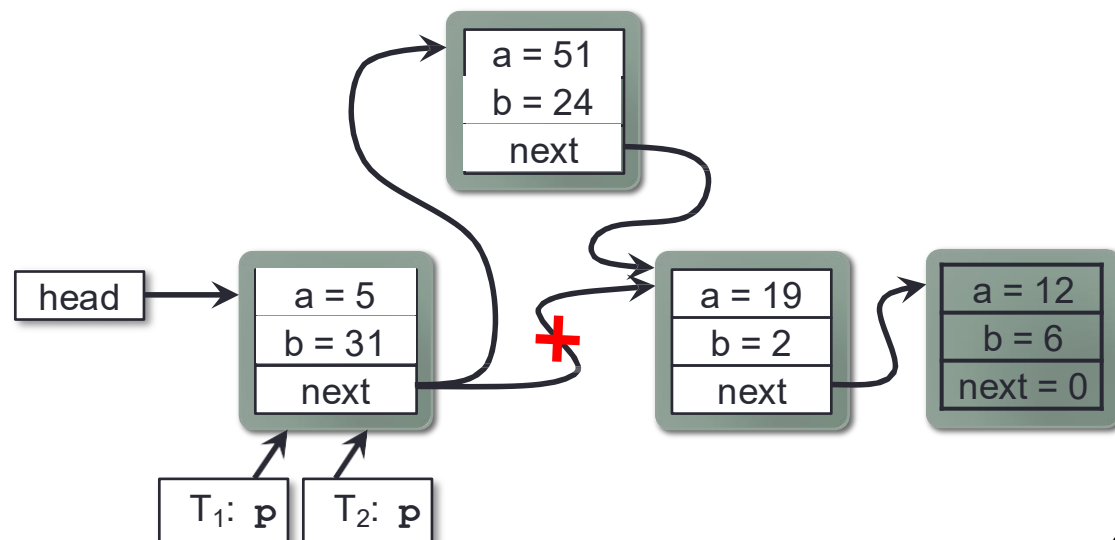
- How does a writer know whether to acquire a read lock or a write lock on a given node?
- Need to acquire a read lock first, examine the node, then release and reacquire a write lock if the node must be altered



Hand-over-hand locking

■ T_2 acquires a read-lock on head so it won't change

- Then T_2 follows head to the first node, and acquires a read lock on this node so it won't change
- When T_2 sees that the new node be placed after the first node, it can acquire a write-lock on the first node (this ensures its changes won't become visible to other threads until the lock is released)
- After T_2 inserts the new node, it can release its locks to allow other threads to see the changes



Hand-over-hand locking

- **How long should each thread hold on to the locks it previously acquired in the list?**
 - If a thread releases locks on nodes after it leaves them, then other threads might change those nodes
- **Does the thread need to be aware of values written by other threads that appear earlier in the list?**
 - What if another thread completely changes the list earlier on?

Hand-over-hand locking

- **If these scenarios are acceptable, then threads can release locks as soon as they leave a node**
 - Often, it's acceptable!

- **If such scenarios are unacceptable, threads can hold on to all locks until they are finished with the list**
 - Ensures that each thread will see a completely consistent snapshot of the list until the thread is finished with its task

Conclusion

- **Even simple changes in how locks are managed can have significant implications**
- **Hand-over-hand locking allows for fine-grained concurrency by minimizing contention for locks**
 - Threads only need to contend for locks on adjacent elements rather than the entire data structure
- **Proper ordering of lock acquisitions and careful management of lock releases are necessary**
 - To prevent deadlocks while still ensuring thread safety and concurrency